LLM log

I utilized GitHub Copilot (I'm granted free access to the Pro version because of my academic status), integrated in VS Code, specifically employing Agent mode to communicate with various AI models - switching between them to double check what the other one created. Precisely, I used *Claude Opus 4.6*, *Claude Haiku* and *GPT-4.3-Codex.* Here's the example of double checking:

can you check the sql queries in the 6 files located in the queries folder, check if the column and table names match the ones in the models.py and tell me if each of queries is correct

models.py:52

Planned queries inspection

I'll inspect all query files and compare every referenced table/column against models.py, then give you a file-by-file correctness report. I'm starting by listing the `queries` folder and loading the relevant files.

Reviewed queries and memory, planned parallel file reads

Examples of some prompts:

i would be interested in making a predicting mechanism for stats regarding overview and tokens. the endpoints such as /trends and input_output_ratio and total_cost are interesting. i would make maybe 3 endpoints and each would return the ml prediction for one of these mentioned endpoints. how does that sound?

Considered user brainstorming

That sounds like a strong and very reasonable bonus scope.

Three predictive endpoints for:

- /overview/total-cost
- /tokens/trends
- /tokens/input-output-ratio

is a clean, defensible design for an exam.

a question regarding efficiency. i notice that every query needs to look up the max value for event_timestamp so that it can further use it. can we somehow make it more efficient? do that query once and then keep it stored in the backend so that the queries are more efficient?

Considered options for optimizing max timestamp retrieval

Good thinking. The `(SELECT MAX(event_timestamp) FROM events)` subquery runs on every single query — that's wasteful since the max timestamp doesn't change within a request (or even between requests, since this is historical data).

Let me check how the API routes call these queries to understand the best place to cache it.

Working

Before allowing the model to create the database tables, I extracted one of each events from the raw telemetry file into Notepad++, formatted them and analysed the contents. Only then I let it proceed with the creation of tables as it intended, with some slight modifications that I recommended. After the tables were loaded with events from the raw file, I made sure to find a couple of entities from each table in that file by looking up their event_id and then manually checked everything was inserted correctly.

Before adding the frontend, i manually tested the API endpoints that retrieve the analysed data, tried to run the same sql command directly in the psql terminal, and also manually examined the data from the tables with simple SELECT * commands, just to get the general picture about whether products of those analytic sql queries were relatable.

```
anjamisan@DESKTOP-7UHDAJN:~$ curl -sS 'http://localhost:8000/api/overview/total-cost?days=365'
[{"date":"2025-12-03","total_cost":82.59429019583138},{"date":"2025-12-04","total_cost":90.07933873297185},{"date":"2025
-12-05","total_cost":126.3672381099774},{"date":"2025-12-06","total_cost":99.20834357777017},{"date":"2025-12-07","total
_cost":112.08801027551458},{"date":"2025-12-08","total_cost":161.41310440196372},{"date":"2025-12-09","total_cost":90.24
088933728619},{"date":"2025-12-10","total_cost":119.98611469124691},{"date":"2025-12-11","total_cost":111.21143288277976
},{"date":"2025-12-12","total_cost":95.03557306144805},{"date":"2025-12-13","total_cost":64.86689847610107},{"date":"202
5-12-14","total_cost":111.34202670806269},{"date":"2025-12-15","total_cost":117.685586601766},{"date":"2025-12-16","tota
```

```
claude_analytics=# SELECT DATE(e.event_timestamp)  as date, SUM(a.cost_usd) as total_cost FROM events e JOIN api_requests a ON
    date     |     total_cost
-------------+--------------------
 2025-12-03 |  82.59429019583136
 2025-12-04 |  90.07933873297182
 2025-12-05 | 126.36723810997748
 2025-12-06 |  99.20834357777002
 2025-12-07 | 112.08801027551456
 2025-12-08 | 161.41310440196378
 2025-12-09 |  90.24088933728615
 2025-12-10 | 119.98611469124691
 2025-12-11 | 111.21143288277977
 2025-12-12 |  95.03557306144806
 2025-12-13 |  64.86689847610107
 2025-12-14 | 111.34202670806275
 2025-12-15 |   117.685586601766
 2025-12-16 |  83.59422810832734
 2025-12-17 |  81.08387306320823
 2025-12-18 | 103.83256161956623
 2025-12-19 | 101.41306157699664
 2025-12-20 |  82.01722182639 04
```

```
e.event_id = a.event_id WHERE e.event_timestamp >= '2026-01-31'::timestamp
- MAKE_INTERVAL(days => 60) GROUP BY date ORDER BY date;
```